



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Carandia Lorenzo Brenda Fernanda

N° de Cuenta: 319018961

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 12 de abril de 2025

CALIFICACIÓN: _____

ILUMINACIÓN II

Actividades realizadas

1. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.

Bloque de código

```
spotLights[3] = SpotLight(1.0f, 0.0f, 0.0f, //LUZ DE COFRE
1.0f, 2.0f,
0.0f,0.0f,0.0f,
0.0f, 0.0f, -1.0f,
1.0f, 0.0f, 0.0f,
5.0f);
spotLightCount++;
```

Creemos nuestra luz tipo spotlights[3] de color rojo, en donde le vamos a pasar la posición de nuestro cofre, y que estará apuntando hacia arriba y abajo

```
glm::vec3 cofrePosition = glm::vec3(0.0f, 3.0f, mainWindow.getmovercarro());
spotLights[3].SetPos(cofrePosition + glm::vec3(1.1f, mainWindow.getrotax(), -30.0f)); // Posición base del cofre
```

Para poder ligar nuestra luz al cofre, le pasamos la posición del coche para que avance con él, además de la posición del cofre al abrir y cerrar la tapa, con SetPos

```
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 22.3f, -55.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
pared.RenderModel();
```

Se dibuja una pared para poder visualizar el cambio de la luz del cofre

Ejecución de código



2. Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight hacia adelante y al retroceder ((mover con teclado hacia dirección de X positiva) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo con alguna bandera asignada por ustedes.

Bloque de código

```
spotLights[1] = SpotLight(0.0f, 0.0f, 1.0f, //LUZ DELANTERA
    1.0f, 2.0f, //atenuacion
    0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, -1.0f, //xDir, yDir, zDir
    1.0f, 0.0f, 0.0f,
    25.0f);
spotLightCount++;

spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f, //LUZ TRASERA
    1.0f, 2.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
    1.0f, 0.0f, 0.0f,
    25.0f);
spotLightCount++;
```

Se crea otra luz tipo spotlight de color amarillo, para poder distinguir el cambio entre luces, las cuales apuntaran al piso.

```
//CONTROL DE LUCES DEL COCHE
// Detectar dirección del movimiento
avanzando = mainWindow.getKeys()[GLFW_KEY_N];
retrocediendo = mainWindow.getKeys()[GLFW_KEY_B];

if (avanzando) {
    luzDelanteraActiva = true; // Delantera SIEMPRE encendida al avanzar
    luzTraseraActiva = false;
}
else if (retrocediendo) {
    luzDelanteraActiva = false;
    luzTraseraActiva = true; // Trasera encendida al retroceder
}

glm::vec3 posCoche = glm::vec3(0.0f, 3.0f, mainWindow.getmovercarro());
spotLights[1].SetPos(posCoche + glm::vec3(0.0f, 2.0f, -5.0f)); // Delantera
spotLights[2].SetPos(posCoche + glm::vec3(0.0f, 2.0f, 5.0f)); // Trasera

// --- Prepara luces a enviar al shader ---
SpotLight lucesActivas[4];
unsigned int contadorLuces = 0;

// Linterna (siempre va primero)
lucesActivas[contadorLuces++] = spotLights[0];

// Añade delantera o trasera
if (luzDelanteraActiva) {
    lucesActivas[contadorLuces++] = spotLights[1];
}
else if (luzTraseraActiva) {
    lucesActivas[contadorLuces++] = spotLights[2];
}

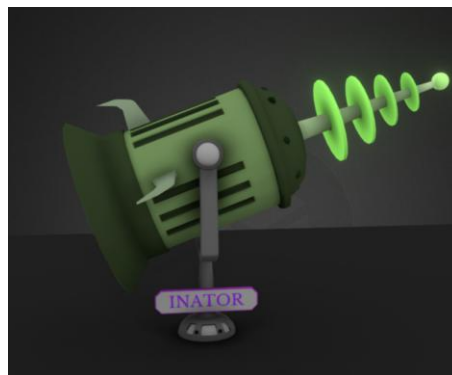
// Luz del cofre (siempre última)
lucesActivas[contadorLuces++] = spotLights[3];
```

Al presionar N (avanzar), activa exclusivamente la luz delantera (spotLights[1]) azul, mientras que al presionar B (retroceder) enciende solo la luz trasera (spotLights[2]) amarillo. La linterna (spotLights[0]) y la luz del cofre (spotLights[3]) permanecen siempre activas. El arreglo temporal (lucesActivas) solo incluye las luces necesarias - primero la linterna, luego la luz activa según el movimiento (delantera o trasera), y finalmente la luz del cofre - asegurando que el shader procese únicamente las luces relevantes mientras mantienen su posición actualizada respecto al movimiento del coche.

Ejecución de código



3.- Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes (no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual)



Elegimos a Inator, un elemento que esta relacionado con nuestro personaje de feria Phineas, además su modelo ya viene texturizado, por lo que no puedo obtener la referencia de las imágenes que se utilizaron solo del modelo como tal. Referencia en bibliografía.

Bloque de código

```
//Declaración de segunda luz puntual
pointLights[1] = PointLight(0.68f, 1.0f, 0.18f, //INATOR
    0.8f, 1.0f,
    65.0f, 15.0f, -23.0f,
    1.0f, 0.09f, 0.032f);
pointLightCount++;

PointLight pointLightsBackup[2]; // Copia de seguridad de las luces originales
pointLightsBackup[0] = pointLights[0]; // Luz 0 original
pointLightsBackup[1] = pointLights[1]; // Luz 1 original
```

Creamos nuestra segunda luz puntual, de color verde

```

//CONTROL DE LAMPARA (Tecla O)
if (mainWindow.getKeys()[GLFW_KEY_O] && !tecla_o) {
    tecla_o = true;
    luz0Encendida = !luz0Encendida;
}
else if (!mainWindow.getKeys()[GLFW_KEY_O]) {
    tecla_o = false;
}

// Control para INATOR (Tecla P)
if (mainWindow.getKeys()[GLFW_KEY_P] && !tecla_p) {
    tecla_p = true;
    luz1Encendida = !luz1Encendida;
}
else if (!mainWindow.getKeys()[GLFW_KEY_P]) {
    tecla_p = false;
}

// Reordenar el arreglo pointLights para que las luces encendidas estén primero
int index = 0;
if (luz0Encendida) {
    pointLights[index++] = pointLightsBackup[0]; // Mover luz 0 al inicio si está encendida
}
if (luz1Encendida) {
    pointLights[index++] = pointLightsBackup[1]; // Mover luz 1 después si está encendida
}

// Actualizar pointLightCount (número de luces encendidas)
pointLightCount = index;

```

*Al presionar **O** se alterna el estado de la primera luz (luz0Encendida) y al presionar **P** se controla la segunda (luz1Encendida), utilizando flags (tecla_o, tecla_p) para evitar activaciones repetidas mientras se mantienen las teclas presionadas. Luego, reorganiza dinámicamente el arreglo pointLights colocando solo las luces encendidas al principio (usando copias de respaldo pointLightsBackup para preservar los valores originales) y actualiza el contador pointLightCount con la cantidad de luces activas.*

```

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(50.0f, 12.0f, -10.0f));
//model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
inator.RenderModel();

```

Mandamos a dibujar a nuestro objeto ya texturizado.

Ejecución de código





Problemas presentados.

En esta práctica no se me presentaron problemas a la hora de ejecutar el código, solo tuve problemas para poder cambiar de luces spotlight, puesto que se apagaban las demás cuando quería cambiar, pero es algo que pude modificar con ayuda.

Conclusiones

Estos ejercicios fueron de gran ayuda para entender cómo funciona la iluminación cuando tenemos distintos tipos de luces y queremos apagarlas y prenderlas para darle un uso en específico, me ayudaron para entender un poco mejor los arreglos y como manejarlos para enviar luces específicas al shader y cuando no, me parecieron de la complejidad adecuada.

Bibliografía

- ✚ Códigos de colores HTML. (s. f.). HTML Color Codes. <https://htmlcolorcodes.com/es/>
- ✚ Sketchfab. (s. f.-a). INATOR (Phineas & Ferb) - Download Free 3D model by Drawnah (@EzequielMunoz). <https://skfb.ly/ox8tC>
- ✚ LearnOpenGL - Light casters. (s. f.-c). <https://learnopengl.com/Lighting/Light-casters>